

1.º Examen parcial – 27/04/2026

1. Tenemos un grafo representado con una matriz de adyacencia A . Dicha matriz tiene únicamente unos y ceros (según si dos vértices son adyacentes, o no). Implementar un algoritmo de **división y conquista** que, dada la matriz A y un valor k , devuelva la cantidad de caminos de longitud k que hay en el grafo correspondiente. Analizar y justificar detalladamente la complejidad del algoritmo implementado. Tener mucho cuidado al analizar la complejidad; es probable que no puedas aplicar el teorema maestro. En dicho caso, como parte del análisis de la complejidad, explicar por qué no es aplicable el teorema (nuevamente, es probable que no puedas aplicarlo).

Recomendamos recordar que:

- $A^m[i][j]$ nos dice la cantidad de caminos de longitud m que hay entre i y j .
- Que la multiplicación de matrices se puede considerar como una operación que consume $\mathcal{O}(n^{\log_2 7})$, siendo n la dimensión de la matriz (cuadrada). Si bien este algoritmo es de división y conquista, no se pide ni es de interés que implementen nada al respecto de esto. Suponé que está disponible la función `multiplicar_matrices(A, B)`.

2. Analicemos la siguiente variante del problema de scheduling que vimos en clase. Tenemos un procesador que puede operar 24 horas al día, todos los días. Tenemos diferentes tareas, donde cada una tiene un horario de inicio y fin fijos. Si se decide realizar una determinada tarea, esta se realiza todos los días. Considerar que algunos trabajos pueden empezar antes de la medianoche y terminar luego de medianoche, y he aquí la diferencia con el problema de scheduling que vimos en clase.

Dada una lista de n tareas, implementar un algoritmo **greedy** que nos devuelva el listado de mayor cantidad de tareas que se puedan realizar, considerando que el procesador solo puede ejecutar una tarea en un determinado momento. Indicar y justificar la complejidad del algoritmo. Justificar por qué es un algoritmo Greedy. ¿El algoritmo da siempre la solución óptima? Si lo hace, justificar, si no dar un contraejemplo.

Por ejemplo, si tenemos las tareas definidas por los siguientes intervalos:

(6 P.M., 6 A.M.), (9 P.M., 4 A.M.), (3 A.M., 2 P.M.), (1 P.M., 7 P.M.)

La solución óptima sería elegir el trabajo de (9 P.M., 4 A.M.) y el de (1P.M., 7 P.M.).

3. Implementar un algoritmo que, por **backtracking**, resuelva el problema del ejercicio 2.
4. Implementar un algoritmo que, por **programación dinámica**, dado un grafo no dirigido y **pesado**, dos vértices s y t , y un número k , determine el costo del camino mínimo entre s y t que utilice **exactamente** k aristas. El camino **puede** repetir vértices (es decir, el camino puede no ser simple). También escribir el algoritmo que permita reconstruir la solución. Indicar y justificar la complejidad del algoritmo implementado (el de programación dinámica, y también el de la reconstrucción).

1.º Examen parcial – 27/04/2026

1. Tenemos un grafo representado con una matriz de adyacencia A . Dicha matriz tiene únicamente unos y ceros (según si dos vértices son adyacentes, o no). Implementar un algoritmo de **división y conquista** que, dada la matriz A y un valor k , devuelva la cantidad de caminos de longitud k que hay en el grafo correspondiente. Analizar y justificar detalladamente la complejidad del algoritmo implementado. Tener mucho cuidado al analizar la complejidad; es probable que no puedas aplicar el teorema maestro. En dicho caso, como parte del análisis de la complejidad, explicar por qué no es aplicable el teorema (nuevamente, es probable que no puedas aplicarlo).

Recomendamos recordar que:

- $A^m[i][j]$ nos dice la cantidad de caminos de longitud m que hay entre i y j .
- Que la multiplicación de matrices se puede considerar como una operación que consume $\mathcal{O}(n^{\log_2 7})$, siendo n la dimensión de la matriz (cuadrada). Si bien este algoritmo es de división y conquista, no se pide ni es de interés que implementen nada al respecto de esto. Suponé que está disponible la función `multiplicar_matrices(A, B)`.

2. Analicemos la siguiente variante del problema de scheduling que vimos en clase. Tenemos un procesador que puede operar 24 horas al día, todos los días. Tenemos diferentes tareas, donde cada una tiene un horario de inicio y fin fijos. Si se decide realizar una determinada tarea, esta se realiza todos los días. Considerar que algunos trabajos pueden empezar antes de la medianoche y terminar luego de medianoche, y he aquí la diferencia con el problema de scheduling que vimos en clase.

Dada una lista de n tareas, implementar un algoritmo **greedy** que nos devuelva el listado de mayor cantidad de tareas que se puedan realizar, considerando que el procesador solo puede ejecutar una tarea en un determinado momento. Indicar y justificar la complejidad del algoritmo. Justificar por qué es un algoritmo Greedy. ¿El algoritmo da siempre la solución óptima? Si lo hace, justificar, si no dar un contraejemplo.

Por ejemplo, si tenemos las tareas definidas por los siguientes intervalos:

(6 P.M., 6 A.M.), (9 P.M., 4 A.M.), (3 A.M., 2 P.M.), (1 P.M., 7 P.M.)

La solución óptima sería elegir el trabajo de (9 P.M., 4 A.M.) y el de (1P.M., 7 P.M.).

3. Implementar un algoritmo que, por **backtracking**, resuelva el problema del ejercicio 2.
4. Implementar un algoritmo que, por **programación dinámica**, dado un grafo no dirigido y **pesado**, dos vértices s y t , y un número k , determine el costo del camino mínimo entre s y t que utilice **exactamente** k aristas. El camino **puede** repetir vértices (es decir, el camino puede no ser simple). También escribir el algoritmo que permita reconstruir la solución. Indicar y justificar la complejidad del algoritmo implementado (el de programación dinámica, y también el de la reconstrucción).